

Matt Pocock's Skills: a Practical Learning Guide

Snapshot studied: [mattpocock/skills](#) commit [c55ee46073ed923f86ce59a5eb3b6d895095d1b7](#) (18 September 2026). This guide covers every [SKILL.md](#) in that snapshot: **38 skills**: 25 promoted daily-use skills, 4 misc skills, and 9 in-progress skills. The promoted public set is described by AI Hero as portable, composable process files that can work across agents; it also presents the recommended engineering spine as clarify → specify → ticket → implement → review.[1][2]

Executive take

This is not a “prompt pack.” It is an opinionated operating system for agent-assisted engineering. Its central bet is sound: agents need repeatable feedback and decision-making processes, not longer one-shot prompts. The highest-value skills form a chain:

[setup](#) → [grill-with-docs](#) → [to-spec](#) → [to-tickets](#) → [implement \(tdd \)](#) → [code-review](#)

Use the chain when work has enough consequence to deserve a written trail. Do **not** force it onto a two-line fix. For a one-session, well-understood task, grill briefly and go to [implement](#); for a genuinely uncertain or cross-session effort, use the full spine. The author's public teaching emphasizes the same pattern: shared understanding first, a spec after the conversation, tracer-bullet tickets, test-first implementation, and ongoing architectural upkeep.[2][3]

What “all skills” means in this repository

Bucket	Count	Status	What it means for you
engineering/ + productivity/	25	Promoted	Daily-use skills, in the README and Claude plugin, with AI Hero docs. Start here.
misc/	4	Unpromoted	Useful niche tools, excluded from the plugin and public skill list. Treat as optional utilities.
in-progress/	9	Public beta	Explicitly unstable and may change or disappear. Try deliberately, not as foundational process.

Invocation: user-invoked skills are orchestration commands a human must call; model-invoked skills are reusable discipline the agent may select when the task fits.[1]

The learning path I recommend

1. **Install selectively, then set up one repository.** [setup-matt-pocock-skills](#) configures tracker, triage labels, and domain-document layout. The official installation options are an editable [npx skills@latest](#)

`add mattpocock/skills` install or the managed Claude plugin; do not install both because that duplicates skills.[1][2]

2. **Learn the 5 core behaviors:** `grill-with-docs` , `to-spec` , `to-tickets` , `tdd` , `code-review` .
3. **Add upkeep:** `diagnosing-bugs` , `improve-codebase-architecture` , `domain-modeling` , `research` .
4. **Use productivity tools when friction appears:** `handoff` , `to-questionnaire` , `wait-what` , `teach` .
5. **Treat beta/misc skills as local experiments.** Do not make a beta workflow your team's process until it survives real work.

The promoted engineering skills (18)

`/setup-matt-pocock-skills`

Solves: unconfigured assumptions about trackers, triage, glossary/ADR locations.

Use when: once per repo before the engineering flow, or when switching trackers.

How it works: inspects the repository, asks for tracker/labels/docs choices, shows drafts, then writes a small Agent Skills block plus `docs/agents/*` configuration.

Pros: makes downstream skills' hidden assumptions explicit; avoids creating `AGENTS.md` when `CLAUDE.md` already exists.

Cons / boundaries: interactive by design; it explores and confirms before writing. It is setup, not project understanding.

Sequence: prerequisite for the engineering system; `code-review` specifically expects its issue-tracker configuration.

`/ask-matt`

Solves: choosing a skill when you know the situation but not the map.

Use when: "what should I run for this?"

How it works: routes among the main flow, codebase health, vocabulary work, phase transitions, and standalone tasks.

Pros: stops random skill invocation and distinguishes repo work from stateless work.

Cons / boundaries: it recommends; it does not execute your project. Keep grilling/spec/ticket work in one context window where possible.

Sequence: router over the rest; use `grill-with-docs` rather than `grill-me` when you have a working repository.

`/grill-with-docs`

Solves: misalignment plus undocumented design vocabulary.

Use when: starting a design or plan in a real codebase.

How it works: wraps `grilling` and `domain-modeling` , producing clarified decisions while updating glossary/ADR material.

Pros: creates a durable shared language instead of a disposable chat.

Cons / boundaries: it is intentionally a thin wrapper; the actual interview and documentation rules live in its dependencies.

Sequence: usual start of repository work; optionally detour through `prototype` , then go direct to `implement` or through `to-spec` and `to-tickets` .

`/to-spec`

Solves: a good conversation that never becomes a buildable, testable contract.

Use when: agreement exists and you want synthesis, not more questions.

How it works: explores the current code, identifies viable testing seams, checks them with you, writes structured user stories/decisions/tests/scope, then publishes to the configured tracker.

Pros: anchors work in behavior and boundaries rather than implementation trivia.

Cons / boundaries: it must not interview; requires tracker and triage configuration; avoids stale paths and incidental code snippets.

Sequence: follows grilling or a prototype; commonly precedes `to-tickets`.

`/to-tickets`

Solves: a spec too large for one fresh agent context or parallel work.

Use when: you need independently executable slices and explicit blockers.

How it works: turns a plan/spec into vertical tracer-bullet tickets, establishes dependencies, quizzes/iterates, and publishes each item to the tracker or local files.

Pros: exposes the unblocked frontier; each ticket is meant to be demoable and small enough for one context.

Cons / boundaries: needs tracker/labels; do not mutate the parent issue; mechanical broad refactors need expand-contract instead of fake vertical slices.

Sequence: `to-spec` → `to-tickets` → implement tickets at the frontier.

`/implement`

Solves: implementation that skips tests, final verification, and review.

Use when: a spec or ticket set is ready.

How it works: builds the requested work, uses `tdd` at agreed seams where possible, repeatedly runs focused checks, runs the full suite, calls `code-review`, and commits.

Pros: makes verification and review part of delivery rather than an optional afterthought.

Cons / boundaries: TDD is conditional on usable pre-agreed seams; this skill assumes the work is already sufficiently described.

Sequence: direct after small, clear grilling; otherwise after `to-spec` → `to-tickets`.

`/tdd`

Solves: plausible agent code with no tight behavioral feedback loop.

Use when: building one behavior or fixing a bug test-first.

How it works: reads domain docs, agrees public seams, then repeats one vertical slice at a time: failing behavior test → minimal implementation → repeat.

Pros: tests survive refactors because they target behavior at public interfaces.

Cons / boundaries: do not write a test before agreeing the seam; rejects tautological/internal tests and horizontal test slicing; refactoring belongs outside red-green.

Sequence: used by `implement`; consult `codebase-design` when the interface/seam is unclear.

`/code-review`

Solves: a change that “looks fine” but either violates standards or misses the originating spec.

Use when: reviewing a branch, PR, work in progress, or a diff since a fixed point.

How it works: validates the fixed point and non-empty diff, locates the spec and standards, then runs isolated parallel reviews: **Standards** and **Spec**.

Pros: two independent axes avoid standards compliance hiding a missed requirement, or vice versa.

Cons / boundaries: needs a fixed point; asks if missing. Without a spec, the Spec axis is explicitly unavailable. Repository standards outrank its Fowler-smell baseline.

Sequence: final stage of `implement`, but valuable standalone.

`/diagnosing-bugs`

Solves: random debugging and untested “fixes.”

Use when: behavior is broken, failing, throwing, regressing, or slow.

How it works: build a tight red-capable loop; reproduce/minimize; propose 3–5 ranked falsifiable hypotheses; instrument one prediction at a time; write regression test before fix if a proper seam exists; verify and clean up.

Pros: forces evidence before theory; minimization shrinks the search space.

Cons / boundaries: never hypothesize before a loop exists; redact secrets. If no loop can be built, stop and request needed access. For performance, measure/bisect rather than log.

Sequence: feeds `improve-codebase-architecture` if no suitable regression-test seam exists.

`/improve-codebase-architecture`

Solves: shallow modules, weak seams, and code that agents struggle to navigate or test.

Use when: weekly-ish upkeep, after a development surge, or around a named pain point.

How it works: targets recently changed hotspots, reads context/ADRs, explores candidates, applies the deletion test, creates a self-contained visual HTML report, then grills the selected candidate.

Pros: prioritizes high-leverage deepening opportunities, with before/after diagrams and test/locality reasoning.

Cons / boundaries: a survey, not a magic mud-removal service. Must not propose interfaces before you choose a candidate; report is temporary, outside the repo.

Sequence: uses `codebase-design`, then `grilling` + `domain-modeling` for the chosen opportunity.

`/codebase-design`

Solves: vague architectural language and accidental shallow-module design.

Use when: shaping module interfaces, seams, testability, or deepening opportunities.

How it works: supplies a strict vocabulary—module, interface, implementation, depth, seam, adapter, leverage, locality—and applies interface-reduction/deletion-test questions.

Pros: gives a shared design lens: lots of behavior behind a small interface.

Cons / boundaries: deliberately prescriptive vocabulary; do not measure depth by line-count ratios or introduce a seam for only one adapter.

Sequence: reference discipline under `tdd` and architecture improvement.

`/domain-modeling`

Solves: overloaded terms, code/conversation mismatch, and ADRs written for trivia.

Use when: terms are fuzzy, `CONTEXT.md` needs work, or a decision may deserve an ADR.

How it works: challenges terms, stress-tests scenarios, checks against code, updates glossary inline, and adds ADRs only for hard-to-reverse, surprising, genuine trade-offs.

Pros: improves naming, navigation, and concise communication across future sessions.

Cons / boundaries: `CONTEXT.md` is a glossary, not an implementation dump, spec, or scratchpad. Create docs lazily.

Sequence: used by `grill-with-docs`, `triage`, `wayfinder`, and architecture improvement.

`/research`

Solves: design decisions made from stale secondary summaries.

Use when: you need documentation/API facts or background investigation before design.

How it works: delegates a background agent to primary docs, source, specs, or APIs; traces claims to owners; saves cited Markdown in repository convention.

Pros: preserves the main session while research proceeds; emphasizes source ownership.

Cons / boundaries: primary sources only; follows existing notes conventions.

Sequence: produces input for `grill-with-docs`; it supplements rather than replaces design.

`/prototype`

Solves: questions that paper plans cannot settle.

Use when: you must validate a state model/logic or compare radically different UI directions.

How it works: first names the exact question. Logic gets a shareable single-page state-machine demo; UI gets multiple route-selectable variants. Validate, record the verdict, then fold only the decision into production work.

Pros: creates runnable evidence, often usable by non-developers.

Cons / boundaries: deliberately throwaway: no tests, broad error handling, or needless abstractions; does not

go on main.

Sequence: optional evidence loop after grilling; `handoff` can bridge its separate directory.

`/wayfinder`

Solves: a large/foggy effort that cannot be honestly planned in one session.

Use when: you need a tracker-backed map of decisions before implementation.

How it works: grills the destination, maps frontier/fog as decision tickets, wires dependencies, dispatches research, resolves one frontier decision per session, updates the map, and graduates fog only when it becomes actionable.

Pros: supports parallel work without pretending uncertainty is resolved.

Cons / boundaries: planning-only by default; do not resolve more than one ticket/session except research; human-in-the-loop decisions require a human.

Sequence: when the route clears, hand off to `to-spec` / `to-tickets`.

`/triage`

Solves: raw issue queues that lack ownership, state, evidence, or usable briefs.

Use when: reviewing incoming issues/PRs or managing an attention queue.

How it works: gathers tracker/code context, detects duplicates/prior rejection, verifies claims, grills if needed, then applies category/state labels, comments, closures, and ready-for-agent briefs.

Pros: small state machine and actionable work queue; checks “already implemented?” before planning.

Cons / boundaries: depends on configured label mapping; comments need the prescribed AI-triage disclaimer; exactly one category and one state role.

Sequence: may invoke `grilling` / `domain-modeling`; can feed implementation.

`/resolving-merge-conflicts`

Solves: line-level conflict resolution that destroys one side's intent.

Use when: a merge or rebase is already in conflict.

How it works: inspects state/history/files, traces each side to commits/PRs/issues, resolves each hunk by intent, tests, stages, and finishes the operation.

Pros: seeks to preserve both intentions rather than mechanically choosing ours/theirs.

Cons / boundaries: must not invent behavior and must not abort. Incompatibility requires choosing the merge goal and recording the trade-off.

Sequence: standalone incident skill.

`/wizard`

Solves: human-only setup steps that agents cannot safely perform (dashboards, secrets, provisioning/cutovers).

Use when: a repeatable manual process needs a guided script.

How it works: inspects context, proposes stages, confirms scope, writes an interactive Bash wizard from a template, validates syntax/ShellCheck, makes it executable, and traces value destinations.

Pros: supports confirmation gates, hidden secret entry, idempotent `.env` updates, and a close-out summary.

Cons / boundaries: do not use it for work the agent can perform; do not invent dashboard steps; do not end-to-end run browser/human-input flows. Ephemeral by default.

Sequence: companion for human execution, not a deployment agent.

The promoted productivity skills (7)

/grill-me

Solves: a plan accepted before its decision tree is resolved. **Use when:** you want an interview without repository documentation. **How:** thin user-facing router to `grilling`. **Pros:** tiny and direct. **Cons:** creates no durable project context. **Sequence:** use `grill-with-docs` instead for repo work.

/grilling

Solves: agents silently filling decision gaps with guesses. **Use when:** sharpening a plan, decision, or idea. **How:** maps a design tree, asks all currently unblocked questions in numbered rounds with recommendations, recomputes the frontier, and stops only once no decision is silently assumed. **Pros:** respects dependencies and separates facts the agent can discover from choices the user must make. **Cons:** no action until shared understanding is confirmed. **Sequence:** core primitive behind `grill-me`, `grill-with-docs`, `triage`, `wayfinder`, and architecture work.

/handoff

Solves: lost context when switching agents/sessions. **Use when:** moving work to a new session. **How:** writes a focused handoff to OS temporary storage, points at existing artifacts, proposes relevant skills, and redacts sensitive material. **Pros:** preserves continuity without duplicating specs/commits/diffs. **Cons:** must remain outside the workspace; not a substitute for real source artifacts. **Sequence:** after any partially completed phase.

/teach

Solves: disconnected one-off explanations when the user needs durable learning. **Use when:** learning a topic over multiple sessions. **How:** maintains mission/resources/records/lessons, researches trusted sources, calibrates difficulty, and creates short interactive HTML lessons with retrieval practice, spacing, and interleaving. **Pros:** mission-linked, cumulative learning. **Cons:** uses the current directory as stateful teaching workspace and must build `RESOURCES.md` before relying on memory. **Sequence:** subsequent lessons use learning records and assets.

/to-questionnaire

Solves: a decision blocked by information held by another person. **Use when:** collecting answers async or in a meeting. **How:** grills the sending context, then writes a clear Markdown questionnaire with answer space, ordered by importance. **Pros:** turns vague "ask them about it" into a usable instrument. **Cons:** asks about recipient/needed result, not the subject; questions must be single-idea. **Sequence:** answers feed later planning/specification.

/wait-what

Solves: an explanation that did not land. **Use when:** immediately after a confusing agent message. **How:** re-pitches it in Simplified Technical English using project ubiquitous language. **Pros:** lowers cognitive load without losing relevant context. **Cons:** must honor `CONTEXT-MAP.md` when several glossaries exist. **Sequence:** recovery/clarification only.

/writing-for-agents

Solves: bloated or stale agent-facing docs that waste context and cause inconsistent behavior. **Use when:** editing skills, `AGENTS.md`, `CLAUDE.md`, or pointed-to documents. **How:** applies progressive disclosure, clear pointers, hierarchy, positive instructions, completion bounds, and pruning. **Pros:** improves predictable behavior while controlling context cost. **Cons:** process guidance, not a guarantee; split files only when the split earns its

load. **Sequence:** a reference tool for agent-facing documentation. AI Hero's companion guidance makes the same case: keep root agent files minimal and point to detailed materials progressively.[4]

Misc skills (4): niche utilities, not the core system

git-guardrails-claude-code

Blocks dangerous Git operations with a Claude Code PreToolUse hook: push, hard reset, forced clean, destructive branch deletion, and working-tree discard. **Use:** when you want preventative guardrails. **Pros:** intercepts before execution and can be project/global. **Cons:** must merge rather than overwrite hooks; customize and test the patterns. **Status:** model-invoked; Claude-specific.

migrate-to-shoehorn

Replaces test-only TypeScript `as` assertions with `@total-typescript/shoehorn` helpers: `fromPartial`, `fromAny`, and sometimes `fromExact`. **Use:** test fixtures are huge or assertions erase useful checking. **Pros:** concise partial fixtures while retaining type-aware alternatives. **Cons:** test code only; carefully distinguish valid partial data from intentionally-invalid inputs. **Status:** model-invoked.

scaffold-exercises

Creates `ai-hero-cli`-conforming course-exercise directories, readmes, stubs, and validates them with its linter. **Use:** authoring AI Hero course exercises. **Pros:** encodes structural/lint rules. **Cons:** highly product-specific. **Status:** model-invoked.

setup-pre-commit

Adds Husky/lint-staged/Prettier plus available typecheck/test commands. **Use:** need commit-time hygiene. **Pros:** formatting first, then checks; detects package manager/config. **Cons:** defaults to npm if unclear and omits absent scripts; assumes Husky v9+. **Status:** model-invoked.

In-progress skills (9): beta, useful but not foundational

/claude-handoff

Starts a fresh background Claude agent with a concise, named handoff prompt. **Use:** continuation should start immediately, not as a saved document. **Pros:** direct continuation and descriptive job identity. **Cons:** Claude CLI-specific; redact secrets because the summary is a prompt. **Sequence:** background alternative to `handoff`.

/implement-spec

Implements an entire spec/ticket dependency graph in one draft-PR branch using worktrees, parallel implementers, a merger agent, review, one fix pass, and cleanup. **Use:** spec + linked tickets already exist. **Pros:** concurrency on the ready frontier. **Cons:** operationally complex; requires clean worktree lifecycle and shared outside-repo exploration notes. **Sequence:** spec/tickets → parallel implementation → `code-review` → fix → ready PR.

/loop-me

Turns recurring life/work patterns into implementable workflow specs over multiple sessions. **Use:** designing a workflow or finding one worth designing. **Pros:** uses a loop lens to surface delegation opportunities. **Cons:**

produces only specs; it may have no AI, schedule, or checkpoint. **Sequence:** invokes `grilling`; `workflows/*.md` is source of truth.

pr

Writes a concise PR body: Summary with the smallest explanatory visual, Evidence, and Merge Danger (one-way/two-way door plus blast radius). **Use:** drafting a PR description. **Pros:** forces evidence and reversibility thinking. **Cons:** no boilerplate theater; use only the visual form that answers the question. **Status:** model-invoked.

/retro

Reviews a coding session and proposes agent-environment improvements across navigation, checks, standards, context, tool use, and information access. **Use:** retrospective process improvement. **Pros:** routes deterministic violations toward automation rather than prose. **Cons:** explicitly a **STUB/design note**, not functional yet; inspect existing CI/checks before proposing a guardrail. **Sequence:** first invokes `writing-for-agents`.

/setup-ts-deep-modules

Configures dependency-cruiser to make package roots public entry points and subfolders private implementation. **Use:** TypeScript repo wants enforced deep-module boundaries. **Pros:** verifies clean pass → intentional deep-import failure → clean pass; allows multiple small entry points. **Cons:** must merge config, not overwrite; no tsconfig/path-alias changes; flat package layer constraint. **Sequence:** uses `codebase-design` vocabulary.

/writing-fragments

Explores raw ideas as append-only Markdown fragments without an outline. **Use:** gathering material before knowing the article. **Pros:** preserves sharp claims, observations, quotes, and leading words. **Cons:** no body headings/order/structure; reread before append. **Sequence:** exploration, then a later writing phase.

/writing-beats

Turns a fixed raw-material pile into an article one coherent “beat” at a time. **Use:** exploration is complete but article route is uncertain. **Pros:** makes conceptual prerequisites and transitions explicit. **Cons:** user chooses each beat; cannot rely on ungrounded concepts; do not write ahead. **Sequence:** exploit fixed raw material.

/writing-shape

Turns read-only raw Markdown into a separate article block-by-block, with explicit format choices. **Use:** raw material exists and you want deliberate article shaping. **Pros:** preserves input, exposes structure/format choices, and refuses invented material. **Cons:** no publishing/platform formatting/frontmatter unless asked; re-read output before every write. **Sequence:** exploit, distinct from fragments’ exploration and beats’ larger narrative units.

Practical decision guide

If you are here...	Run this	Avoid
"I know roughly what I want, but the decisions are mushy."	grill-with-docs (repo) or grill-me (stateless)	Asking the agent to write a plan immediately
"We agree; make it buildable."	to-spec	Re-grilling the same decisions
"This won't fit in one agent session."	to-tickets	Horizontal tasks such as "make all types first"
"We do not know whether this model/UI will work."	prototype	Turning the experiment into production code
"Implement this ready, bounded behavior."	implement + tdd	Tests that assert private implementation
"Something is broken/slow."	diagnosing-bugs	Hypotheses before a reproducible feedback loop
"The agent has made the codebase harder to change."	improve-codebase-architecture	Big-bang rewrites or treating the report as an automatic fix
"The diff is ready."	code-review	One-axis 'looks clean' review
"A person must perform a secret/dashboard step."	wizard	An agent pretending to execute it
"We keep losing session context."	handoff	Copying every artifact into a giant prompt

Strengths, weaknesses, and adoption advice

Strengths

- **Composable rather than monolithic.** Each skill has a narrow job and explicit handoffs.[1][2]
- **Decision quality before code.** Grilling, domain modeling, specs, and tracer bullets target the most common agent failure: confidently building the wrong thing.[1][3]
- **Feedback-first implementation.** TDD, diagnosis, and review make evidence part of the loop.[1][3]
- **Architecture is a first-class maintenance task.** Deep modules and clean seams are especially valuable with agents because they improve both navigation and testing.[1][3]
- **Progressive disclosure.** The system keeps specialized process material available without putting all rules in a giant root instruction file.[2][4]

Weaknesses / trade-offs

- **Process overhead is real.** The full flow is overkill for trivial changes. Use the shortest path that still gives you confidence.
- **Tracker and documentation discipline are prerequisites.** The engineering flow becomes clumsy if the repository has no agreed tracker/labels/docs arrangement.
- **Some skills are tool- or ecosystem-specific.** Claude hooks, AI Hero exercise layout, and TypeScript dependency-cruiser are not portable defaults.

- **Beta is beta.** The in-progress bucket is explicitly not the stable contract; `retro` is not functional.
- **Skills encode a philosophy.** Deep modules, glossary/ADR behavior, tracer bullets, and strict terminology may not fit every codebase unchanged. Adapt the process, but preserve the underlying feedback loops.

A pragmatic rollout

Week 1: install/edit only `setup-matt-pocock-skills`, `grill-with-docs`, `to-spec`, `tdd`, and `code-review`. Use them on one meaningful feature.

Week 2: add `to-tickets`, `diagnosing-bugs`, and `handoff`; compare rework and review findings against the previous week.

Week 3+: introduce `domain-modeling`, `research`, and `improve-codebase-architecture`; trial beta/misc tools only where they solve a concrete repeated pain.

Do not copy the whole repository blindly into every project. The point is to make the agent's process fit your standards—not to replace your judgment with somebody else's workflow.[1][2]

Sources

[1] <https://github.com/mattpocock/skills>

[2] <https://www.aihero.dev/skills>

[3] <https://www.aihero.dev/5-agent-skills-i-use-every-day>

[4] <https://www.aihero.dev/a-complete-guide-to-agents-md>